

第2章 关系数据库

2.4 关系代数 (基本运算)

(2026 春季)



2.4 关系代数（基本运算）

- 一. 导入：关系代数概述
- 二. 关系代数的基本运算
- 三. 关系代数基本运算的应用
- 四. 基本运算小结

一. 关系代数概述

❑ 关系代数

❑ 关系运算 与 关系操纵

➤ (2.4.1) 传统的集合运算 与 (2.4.2) 专门的关系运算

➤ 基本运算 与 扩充运算

❑ 使用的记号

❑ 关系模式的语义

➤ 关系模式的相容(compatible) / 同类关系

❑ 赋值运算

❑ 算术比较运算 & 逻辑运算

□ 关系代数

- 是一种抽象的查询语言，它用关系运算来表达对关系的操作。
- 在关系代数中，运算对象是‘关系’，运算结果亦为‘关系’

□ 关系代数的运算符有两类：传统的集合运算符 和 专门的关系运算符

- 传统的集合运算是从关系的“水平”方向，即行的角度进行
- 专门的关系运算不仅涉及行而且涉及列

□ 关系代数表达式

- 以关系为运算对象，使用关系运算经有限次复合后形成的表达式称为‘关系代数表达式’
- 可以用关系代数表达式来表示关系模型上的数据操纵（包括数据查询、数据插入、删除和修改操作）

□ 常用的关系操作

- 集合操作：并、交、差、笛卡尔积
- 专门的关系操作
 - 选择、投影
 - 连接（包括 θ -连接，自然连接，外连接等）
 - 除

□ 选择、投影、并、差、笛卡尔积是5种基本操作

□ 关系操作的特点：集合操作

- 操作对象和操作结果都是关系（关系是元组的集合）

关系代数 3：关系运算（传统集合运算 & 专门关系运算）

关系运算		运算符	操作符	
集合运算	并	$R \cup S$	$R \text{ union } S$	
	差	$R - S$	$R \text{ except } S$	
	交	$R \cap S$	$R \text{ intersect } S$	
	笛卡尔积	$R \times S$	$R \text{ product } S$	
专门的关系运算	选择		$\sigma_F(R)$ $R \text{ where } F$	
	投影		$\pi_{A_1, A_2, \dots, A_n}(R)$ $R[A_1, A_2, \dots, A_n]$	
	连接	θ -连接	$R \bowtie_F S$	$(R \text{ product } S) \text{ where } F$
		自然连接	$R \bowtie S$	$R \text{ join } S$
		外连接	$R \Join S$	$R \text{ outer join } S$
		左外连接	$R \Joinleft S$	$R \text{ left outer join } S$
		右外连接	$R \Joinright S$	$R \text{ right outer join } S$
	除		$R \div S$ $R \text{ divideby } S$	

关系代数 4：关系运算（基本运算 & 扩充运算）

	关系运算	运算符	操作符
基本运算	并	$R \cup S$	$R \text{ union } S$
	差	$R - S$	$R \text{ except } S$
	笛卡尔积	$R \times S$	$R \text{ product } S$
	选择	$\sigma_F(R)$	$R \text{ where } F$
	投影	$\pi_{A_1, A_2, \dots, A_n}(R)$	$R[A_1, A_2, \dots, A_n]$
扩充运算	交	$R \cap S$	$R \text{ intersect } S$
	θ -连接	$R \bowtie_F S$	$(R \text{ product } S) \text{ where } F$
	自然连接	$R \bowtie S$	$R \text{ join } S$
	外连接	$R \Join S$	$R \text{ outer join } S$
	左外连接	$R \Joinleft S$	$R \text{ left outer join } S$
	右外连接	$R \Joinright S$	$R \text{ right outer join } S$
	除	$R \div S$	$R \text{ divideby } S$

关系代数 5：关系操纵的表示

□ 对于关系代数中的每一种运算符，都需要从以下两个方面去掌握：

① 运算的执行条件

- 对关系运算的正确使用，确保关系代数表达式可以合法计算

② 运算的执行结果

- 结果关系的关系模式是什么？
- 结果关系中的元组是什么？
- 结果关系的关系模式和结果元组的语义是否准确反映了用户的访问要求？

□ 不论是哪一个关系（被访问的访问对象、访问结果，或者是访问过程中所生成的中间关系），都需要明确其‘关系模式’和‘元组集合’是什么。

- ❑ 基本符号
- ❑ 关系与关系模式的表示
- ❑ 元组连接的表示
- ❑ 象集

关系代数中使用的记号 1

符号	含义
$A, B, C, \dots$ $A_1, A_2, \dots$	前面的大写字母表示属性名、小写字母表示属性值。 例如：用 A 表示属性名，用 a 或 $a_1, a_2, \dots$ 表示属性 A 的值
$X, Y, Z, \dots$	后面的大写字母表示关系中的一个属性组/属性集
$R, S, T, \dots$	中间的大写字母表示关系名，代表一个关系，或该关系的一个实例
$R(U)$	关系模式：关系名 R ，关系中的属性集合 U
$head(R)$	关系 R 中所有属性名的集合(关系模式的另一种表示方式)
$r, s, t, \dots$ r_1, r_2, r_3	关系中的元组
$r_1[A]$	元组 r_1 在属性(集) A 上的取值

使用的记号 2

- 设有一个 n 目关系 $R(A_1, A_2, \dots, A_n)$: R 是关系名, $A_1, A_2, \dots, A_n$ 是属性名
 - 它的一个关系 (实例) 设为 R
 - 关系模式可以表示为 $R(A_1, A_2, \dots, A_n)$ 或 $head(R) = \{A_1, A_2, \dots, A_n\}$
 - $t \in R$ 表示 t 是关系 R 中的一个元组
 - $t[A_i]$ 则表示元组 t 中相应于属性 A_i 的一个分量 (元组 t 在属性 A_i 上的值)
 - 若 $A = \{A_{i_1}, A_{i_2}, \dots, A_{i_k}\}$, 其中 $A_{i_1}, A_{i_2}, \dots, A_{i_k}$ 是来自于 $A_1, A_2, \dots, A_n$ 中的属性, 则 A 称为‘属性列’或‘属性组’或‘属性集’。
 - $t[A] = (t[A_{i_1}], t[A_{i_2}], \dots, t[A_{i_k}])$ 表示元组 t 在属性组 A 上诸分量的集合
 - $\bar{A}$ 则表示从 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_{i_1}, A_{i_2}, \dots, A_{i_k}\}$ 后剩余的属性组

使用的记号 3: 元组的连接

□ R 为 n 目关系, S 为 m 目关系

➤ $t_r \in R, t_s \in S$, $\widehat{t_r t_s}$ 称为元组的连接

➤ $\widehat{t_r t_s}$ 是一个 $n + m$ 列的元组

- 前 n 个分量来自于 R 中的元组 t_r
- 后 m 个分量来自于 S 中的元组 t_s

➤ 为方便表示, 以后用 (t_r, t_s) 来表示元组的连接。

使用的记号 4: 象集 (Images Set)

□ 给定一个关系 $R(X, Z)$

➤ X 和 Z 为属性组

➤ x_0 为属性组 X 定义域上的一个值

➤ x_0 在 R 中的 **象集** 定义如下:

$$Z_{x_0} = \{ t[Z] \mid t \in R \text{ 且 } t[X] = x_0 \}$$

➤ 它表示 R 中属性组 X 上值为 x 的诸元组在 Z 上分量的集合

象集的例子：图2.3

□ x_1 在 R 中的象集

$$Z_{x_1} = \{ z_1, z_2, z_3 \}$$

□ x_2 在 R 中的象集

$$Z_{x_2} = \{ z_2, z_3 \}$$

□ x_3 在 R 中的象集

$$Z_{x_3} = \{ z_1, z_3 \}$$

R	
X	Z
x_1	z_1
x_1	z_2
x_1	z_3
x_2	z_2
x_2	z_3
x_3	z_1
x_3	z_3

象集的例子：图2.4

□ 在关系SC中，存在三个课程号值

➤ 课程号 1 在SC中的象集为

$$Z_{Cno=1} = \{ (201215121, 92) \}$$

➤ 课程号 2 在SC中的象集为

$$Z_{Cno=2} = \{ (201215121, 85), \\ (201215122, 90) \}$$

➤ 课程号 3 在SC中的象集为

$$Z_{Cno=3} = \{ (201215121, 88), \\ (201215122, 80) \}$$

(c) 选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

❑ 关系模型中的命名规则

- 通常遵循“同名同义、异名异义”

❑ 关系模式的相容 (compatible)

- 相容表 (compatible tables)
- 同类关系

关系模式的语义 1: 关系的命名规则

- ❑ 在同一个数据库（模式）中，关系名互不相同
- ❑ 在不同的数据库（模式）之间，允许存在同名、但互不相关的关系
- ❑ 在同一个数据库（模式）中
 - 用不同的关系名来表示不同的关系
 - 不同的关系，其关系模式可能相同、也可能不同
 - 关系模式相同的两个关系，其元组集合可能相同、也可能不同

关系模式的语义 2: 属性的命名规则

- ❑ 在同一个关系中：属性名互不相同，不同的属性可以有相同的属性域
- ❑ 在不同的关系之间
 - 一般通过属性名来确定一个属性的语义，并遵循下述约定：
 - **同名同义**：‘属性名相同’表示具有‘相同的属性域和相同的语义’
 - **异名异义**：‘属性名不同’表示具有‘不同的属性域或不同的语义’
 - 在后续的关系代数表示中，均采用“**同名同义、异名异义**”的约定
 - 但是，属性名也仅仅是用于标识关系中某一系列的标识符，在同一个关系中，不同的列有着不同的属性名。在不同关系之间，也有可能存在属性的‘**同名异义**’和‘**同义异名**’现象，这个时候可以通过赋值运算对关系中的属性进行重命名，使其符合“同名同义、异名异义”的约定。

关系模式的语义 3: 相容表 (compatible tables)

□ 如果关系R和关系S的关系模式满足下述要求，则称关系R和S是相容的（或者称关系R和S是‘相容表’、‘同类关系’、相同的模式）

① 相同的目（两者具有相同的属性数）

② 关系R和S中的属性一一对应、且对应属性之间有着‘相同的域’和‘相同的语义’（确保在对应属性的属性值之间可以进行有意义的逻辑比较）

□ 一般情况下，依据‘同名同义、异名异义’的约定，可以按照属性名来建立属性之间的一一对应，但是也可能存在以下例外：

- 属性名相同，却有着‘不同的属性域’或‘不同的语义’（同名异义）
- 属性名不同，却有着‘相同的属性域’和‘相同的语义’（异名同义）
- (属性名相同或不同)在对应属性之间，‘属性域相同’但‘语义不同’

关系模式的语义 4：相容表的判断

□ 有如下所示的五个关系，其中：

- R 与 S_1 具有相同的模式（相同的目 n 、相应的属性‘同名同义’）
- R 与 S_2 具有不同的模式（属性 A 的‘同名异义’）
- R 与 S_3 具有相同的模式（列的无序性）
- R 与 S_4 可以被认为是不同的模式（属性的‘异名异义’），也可以被作为相同的模式（相同的目、相应列对应相同的域）

A	B	C
a_1	b_1	c_1
a_1	b_2	c_3
a_2	b_1	c_2

关系 R

A	B	C
a_1	b_1	c_1
a_1	b_1	c_2
a_1	b_2	c_3
a_3	b_2	c_3

关系 S_1

A	B	C
1	b_1	c_1
1	b_1	c_2
1	b_2	c_3
3	b_2	c_3

关系 S_2

A	C	B
a_1	c_1	b_1
a_1	c_2	b_1
a_1	c_3	b_2
a_3	c_3	b_2

关系 S_3

A	B	D
a_1	b_1	c_1
a_1	b_1	c_2
a_1	b_2	c_3
a_3	b_2	c_3

关系 S_4

- ❑ 赋值运算符 $:=$
- ❑ 赋值运算的两种使用方式
- ❑ 赋值运算的作用

关系代数中的‘赋值’运算

□ 赋值运算的一般表示方法: $R(A_1, A_2, \dots, A_n) := \langle expression \rangle$

➤ $:=$ 是赋值运算符

➤ 右边的 $\langle expression \rangle$ 是一个关系代数表达式（用于表示在关系数据库上的数据操纵请求），或者是一个当前存在的关系；

➤ 左边的 $R(A_1, A_2, \dots, A_n)$ 是 $\langle expression \rangle$ 的计算结果（即数据操纵返回的结果关系）

上述的赋值表达式可以理解为：

- ① 为右边的数据操纵请求定义了一个查询关系，用以保存其操纵结果；
- ② R 是该查询关系的关系名， $A_1, A_2, \dots, A_n$ 是查询关系的属性名；
- ③ 查询关系也可以被作为访问对象，参与后续关系代数表达式的表示。

赋值运算的使用方式

□ 方式一： $R(A_1, A_2, \dots, A_n) := \langle expression \rangle$

- 计算右边的关系代数表达式 $\langle expression \rangle$ ，并将其结果关系保存下来形成临时的查询关系 R
- 查询关系中的属性名，可以采用它们在右边表达式中的原属性名，也可以对查询关系中的部分属性进行重命名，但必须保证在查询关系 R 中不会出现属性同名现象。

□ 方式二： $R := \langle expression \rangle$

- 将右边的关系代数表达式 $\langle expression \rangle$ 的计算结果保存下来形成临时的查询关系 R
- R 是查询关系的名字，其属性名直接沿用它们在右边表达式中的原属性名，但必须保证在查询关系 R 中不会出现属性同名现象。

赋值运算的用途

1. 对一个关系及关系中的属性进行重命名 (alias), 以方便‘关系的自连接’或者复杂查询的表示。
 - 表的赋值: $S := R$
 - 简单的‘表的赋值’操作, 可用于为关系 R 另起一个别名 (alias), 或者理解为: 定义了一个与关系 R ‘完全相同’的中间关系 S (关系名不同, 但具有相同的模式模式和元组集合)
 - 也可以在赋值表达式中, 对生成的中间关系 S 中的属性进行重命名 (类似于前一页的‘方式一’)
 - 通常用于‘关系自连接’的表示, 或者有目的地对关系中的某些属性进行重命名!
2. 赋值运算也可以用来保存计算的‘中间结果’, 方便复杂查询的表示。

算术比较运算 与 逻辑运算

□ 算术比较运算

- 大于 $>$, 大于等于 $\geq$, 小于 $<$, 小于等于 $\leq$, 等于 $=$, 不等于 $<>$
- 也可以用 $\neq$ 来表示不等比较 $<>$

□ 逻辑运算

逻辑运算	运算符	操作符	示例
与	$\wedge$	<i>and</i>	$F_1 \wedge F_2$, F_1 <i>and</i> F_2
或	$\vee$	<i>or</i>	$F_1 \vee F_2$, F_1 <i>or</i> F_2
非	$\neg$	<i>not</i>	$\neg F$, <i>not</i> F

2.4 关系代数（基本运算）

- 一. 导入：关系代数概述
- 二. 关系代数的基本运算
- 三. 关系代数基本运算的应用
- 四. 基本运算小结

二. 关系代数的基本运算

关系运算		运算符	操作符
基本运算	并	$R \cup S$	$R \text{ union } S$
	差	$R - S$	$R \text{ except } S$
	笛卡尔积	$R \times S$	$R \text{ product } S$
	选择	$\sigma_F(R)$	$R \text{ where } F$
	投影	$\pi_{A_1, A_2, \dots, A_n}(R)$	$R[A_1, A_2, \dots, A_n]$

□ 并 (Union)

➤ R 和 S

- 具有相同的目 n (即两个关系都有 n 个属性)
- 相应的属性取自同一个域

➤ $R \cup S$ 的运算结果

- 仍为 n 目关系, 由属于 R 或属于 S 的元组组成

$$R \cup S = \{ t \mid t \in R \vee t \in S \}$$

➤ 并运算 满足 交换律 和 结合律

- 交换律: $R \cup S = S \cup R$
- 结合律: $(R \cup S) \cup T = R \cup (S \cup T)$

‘并’ 运算 2

□ the set of tuples in the relation $T = R \cup S = \{ t \mid t \in R \vee t \in S \}$

/ 首先将关系 R 中的元组加入结果关系 T */*

$T := R$

for each tuple $t_s \in S$

{

/ 确保结果关系 T 中元组的唯一性 */*

if ($t_s \notin R$)

then add tuple t_s into T

}

return T

□ 差 (Difference)

➤ R 和 S

- 具有相同的目 n
- 相应的属性取自同一个域

➤ $R - S$ 的运算结果

- 仍为 n 目关系，由属于 R 而不属于 S 的所有元组组成

$$R - S = \{ t \mid t \in R \wedge t \notin S \}$$

➤ 差运算 不满足 交换律 和 结合律

- $R - S \neq S - R$
- $(R - S) - T \neq R - (S - T)$

‘差’ 运算 2

□ the set of tuples in the relation $T = R - S = \{ t \mid t \in R \wedge t \notin S \}$

```
/* 将结果关系  $T$  初始化为空集 */
```

```
 $T := \{ \}$ 
```

```
for each tuple  $t \in R$ 
```

```
{
```

```
    /* 仅由属于关系  $R$  但不属于关系  $S$  的元组组成结果关系  $T$  */
```

```
    if ( $t \notin S$ )
```

```
        then add tuple  $t$  into  $T$ 
```

```
}
```

```
return  $T$ 
```

并运算&差运算：图2.2

R

A	B	C
a_1	b_1	c_1
a_1	b_2	c_2
a_2	b_2	c_1

$R \cup S$

A	B	C
a_1	b_1	c_1
a_1	b_2	c_2
a_2	b_2	c_1
a_1	b_3	c_2

S

A	B	C
a_1	b_2	c_2
a_1	b_3	c_2
a_2	b_2	c_1

$R - S$

A	B	C
a_1	b_1	c_1

$S - R$

A	B	C
a_1	b_3	c_2

笛卡尔积 1

❑ 严格地讲应该是广义的笛卡尔积 (Extended Cartesian Product)

❑ 设有两个关系：

- R : n 目关系, k_1 个元组
- S : m 目关系, k_2 个元组

❑ 笛卡尔积 $R \times S$

➤ 结果关系的关系模式: $(n + m)$ 列

- 前 n 列来自于关系 R , 后 m 列来自于关系 S

➤ 结果关系中的行: $(n + m)$ -元组, 共有 $k_1 \times k_2$ 个结果元组

- 结果元组的前 n 列来自于关系 R 的一个元组
- 结果元组的后 m 列来自于关系 S 的一个元组

➤ 可符号化表示为:

$$R \times S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \} = \{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \}$$

笛卡尔积 2

- 任意两个关系，都可以进行笛卡尔积运算
 - 设： R 是 n 目关系， S 是 m 目关系
 - 笛卡尔积 $R \times S$ 的计算结果是一个 $(n + m)$ 目关系（假设为 T ）

```
/* 将结果关系 $T$ 初始化为空集 */  
 $T := \{ \}$   
for each tuple  $t_r \in R$   
{  
    for each tuple  $t_s \in S$   
    {  
        add tuple  $(t_r, t_s)$  into  $T$   
    }  
}  
return  $T$ 
```

❑ 笛卡儿积 满足 交换律 和 结合律

➤ $R \times S = S \times R$

➤ $(R \times S) \times T = R \times (S \times T)$

❑ 如果在关系R和S中存在同名的属性，在执行它们之间的笛卡尔积运算时，将被认为是两个不同的属性。

❑ 在笛卡尔积的结果关系中，对每一对同名属性，必须至少对其中的一个进行换名（确保在结果关系中，属性名是互不相同的）。

笛卡尔积 4: 图2.2 (f)

R		
A	B	C
a_1	b_1	c_1
a_1	b_2	c_2
a_2	b_2	c_1

S		
A	B	C
a_1	b_2	c_2
a_1	b_3	c_2
a_2	b_2	c_1

$R \times S$					
R.A	R.B	R.C	S.A	S.B	S.C
a_1	b_1	c_1	a_1	b_2	c_2
a_1	b_1	c_1	a_1	b_3	c_2
a_1	b_1	c_1	a_2	b_2	c_1
a_1	b_2	c_2	a_1	b_2	c_2
a_1	b_2	c_2	a_1	b_3	c_2
a_1	b_2	c_2	a_2	b_2	c_1
a_2	b_2	c_1	a_1	b_2	c_2
a_2	b_2	c_1	a_1	b_3	c_2
a_2	b_2	c_1	a_2	b_2	c_1

图2.4：基础关系 1

❑ 学生-课程数据库：

学生关系Student、课程关系Course、选修关系SC

(a) 学生关系 Student

学号 Sno	姓名 Sname	性别 Ssex	年龄 Sage	所在系 Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

图2.4：基础关系 2

(b) 课程关系 **Course**

课程号 Cno	课程名 Cname	先行课 Cpno	学分 Ccredit
1	数据库	5	4
2	数学		2
3	信息系统	1	4
4	操作系统	6	3
5	数据结构	7	4
6	数据处理		2
7	PASCAL语言	6	4

图2.4：基础关系 3

(c) 选修关系 SC

学号 Sno	课程号 Cno	成绩 Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

选择 1

❑ 选择 (Selection) 又称为限制 (Restriction)

❑ 选择运算符的含义: $\sigma_F(R)$

选择运算也可表示为: $R \text{ where } F$

➤ 在关系 R 中选择满足给定条件 F 的诸元组

$$\sigma_F(R) = \{ t \mid t \in R \wedge F(t) = \text{true} \}$$

➤ F : 选择条件, 是一个逻辑表达式, 取值为 逻辑真 true 或 逻辑假 false

● 基本形式: $X_1 \theta Y_1$

– X_1 和 Y_1 是关系 R 中的属性名或常量, 但不能都是常量

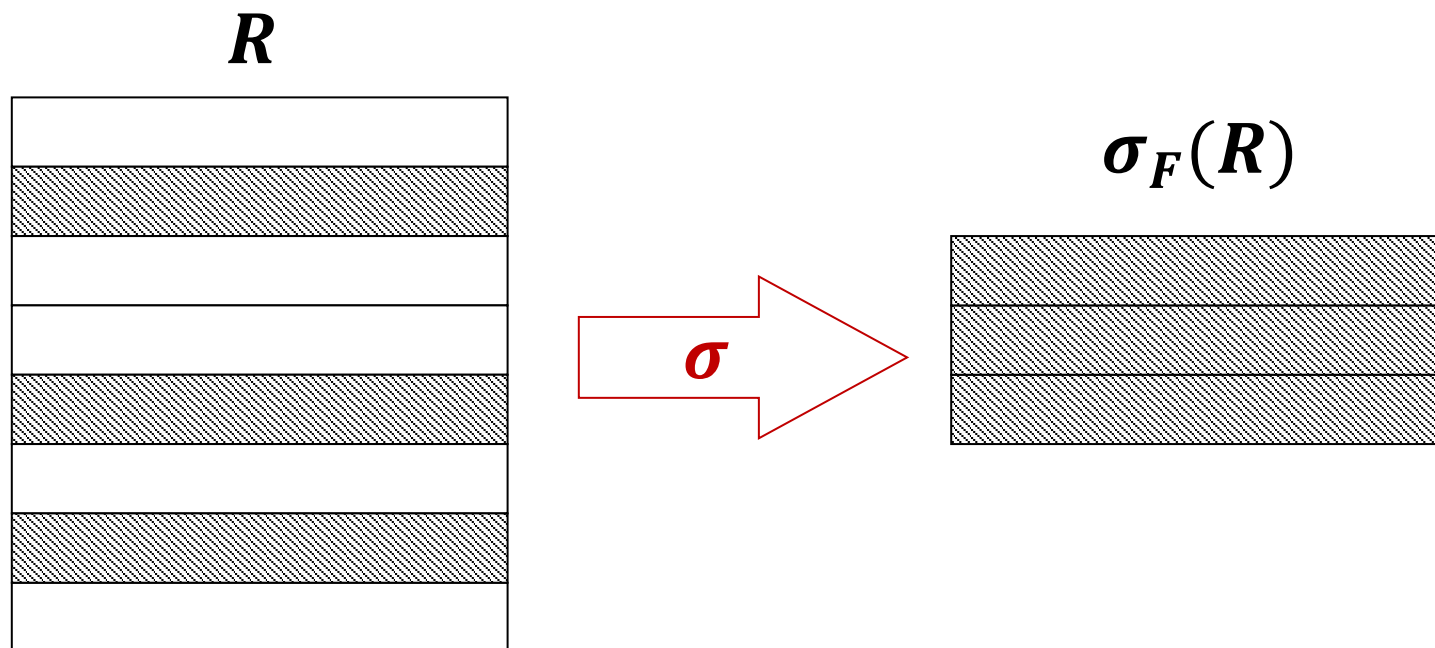
– θ 是比较运算符, 它可以是 $>, \geq, <, \leq, =, <>$ (也可以用 $\neq$ 来表示不等比较 $<>$)

● 复合形式

● 在基本的选择条件上可以进一步进行逻辑运算 (由若干个选择条件通过逻辑运算符 (与、或、非) 经有限次组合而成)

选择 2

- 选择运算是从关系 R 中选取使逻辑表达式 F 为真的元组，是从行的角度进行的运算



- 在关系 R 中，带有阴影背景的元组，将其值代入到选择运算中能够使条件表达式 F 成立，其他元组被代入到选择条件 F 中不成立。

□ 选择: $\sigma_F(R)$

- 结果关系模式: 与关系R相同
- 结果关系元组: 能够使选择条件F成立的关系R中的元组

```
T := { } /* 初始化结果元组集合 */
for each tuple t ∈ R
{
    /* 将元组t的属性值代入到条件表达式F中进行计算 */
    if F(t) = true
    then add tuple t into T
}
return T
```

选择 3

□ 例2.4: 查询信息系 (IS系) 全体学生。

➤ 关系代数表达式为:

$$\sigma_{Sdept = 'IS'} (Student)$$

➤ 结果: 图2.5(a)

Sno	Sname	Ssex	Sage	Sdept
201215125	张立	男	19	IS

➤ 也可表示为: *Student where Sdept = 'IS'*

选择 4

❑ 例2.5：查询年龄小于20岁的学生。

➤ 关系代数表达式为： $\sigma_{Sage < 20}(Student)$

➤ 结果：图2.5(b)

Sno	Sname	Ssex	Sage	Sdept
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

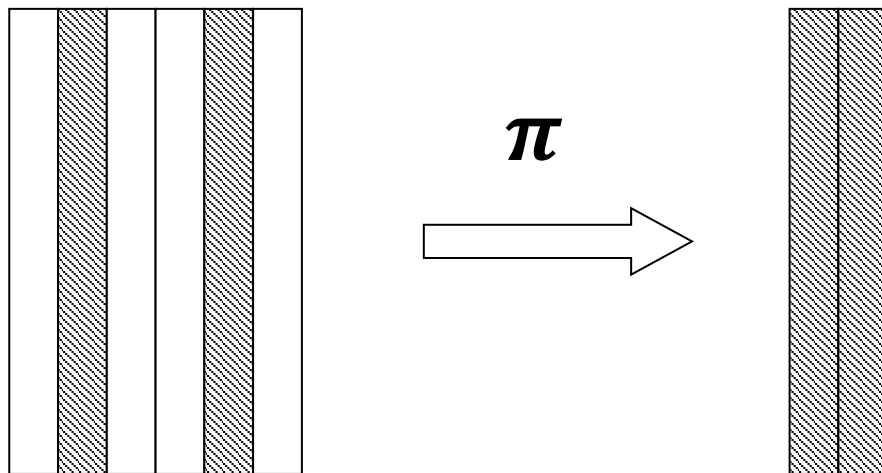
➤ 也可表示为：*Student where Sage < 20*

□ 投影 (Projection)

- 从 R 中选择出若干属性列组成新的关系 (A 是 R 中的属性组)

$$\pi_A(R) = \{ t[A] \mid t \in R \}$$

- 投影操作主要是从列的角度进行运算



- 投影之后不仅过滤了原关系中的某些列，而且还可能产生重复的投影结果元组 (需要消除那些重复出现的行)

投影 2

□ 投影: $\pi_A(R)$

投影运算也可表示为: $R[A]$

➤ 略去关系 R 中的某些列, 并重新安排结果关系中列的排列次序的运算

➤ 设关系 R 有 n 个属性 $A_1, A_2, \dots, A_n$, 在其中 m 个属性 $A_{i_1}, A_{i_2}, \dots, A_{i_m}$ 上的投影运算可以表示为: $\pi_{A_{i_1}, A_{i_2}, \dots, A_{i_m}}(R)$ 或表示为: $R[A_{i_1}, A_{i_2}, \dots, A_{i_m}]$

其中: $A_{i_j} \in \{A_1, A_2, \dots, A_n\} (j = 1, 2, \dots, m)$

➤ 运算结果

- 是一个由 $A_{i_1}, A_{i_2}, \dots, A_{i_m}$ 所组成的 m 元关系
- 关系 R 中的每个元组 t 在 $A_{i_1}, A_{i_2}, \dots, A_{i_m}$ 这 m 个属性上的取值构成结果关系中的一个元组
- 用集合表示法表示为: $\pi_{A_{i_1}, A_{i_2}, \dots, A_{i_m}}(R) = \{t[A_{i_1}, A_{i_2}, \dots, A_{i_m}] \mid t \in R\}$

□ 投影: $\pi_A(R)$

- 结果关系模式: 仅含有属性组 A 中的属性
- 结果关系元组: 由关系 R 中的元组投影而成

```
 $T := \{ \}$  /* 初始化结果元组集合 */  
for each tuple  $t \in R$   
{  
    /* 投影过程中可能产生重复的结果元组  $t[A]$  */  
    if  $t[A] \notin T$   
        then add tuple  $t[A]$  into  $T$   
}  
return  $T$ 
```

□ 例2.6: 查询学生的姓名和所在系。

➤ 即求Student关系上学生姓名和所在系两个属性上的投影

$$\pi_{Sname, Sdept}(Student)$$

➤ 结果: 图2.6(a)

Sname	Sdept
李勇	CS
刘晨	CS
王敏	MA
张立	IS

➤ 也可表示为: $Student[Sname, Sdept]$

❑ 例2.7：查询学生关系Student中都有哪些系。

$$\pi_{Sdept}(Student)$$

➤ 结果：需要消除重复出现的院系名称（图2.6(b)）

Sdept
CS
IS
MA

学生关系 Student

Sno	Sname	Ssex	Sage	Sdept
201215121	李勇	男	20	CS
201215122	刘晨	女	19	CS
201215123	王敏	女	18	MA
201215125	张立	男	19	IS

➤ 也可表示为： *Student* [*Sdept*]

单个关系上的数据操纵：选择 & 投影

□ 可以结合使用投影和选择运算来实现单张表中的数据查询操作：

$$\pi_A(\sigma_F(R))$$

1) 先根据选择条件F对关系R进行选择运算

- 选出符合条件的元组

2) 再对选择运算的结果关系进行投影运算

- 确定最终结果关系的关系模式
- 对符合选择条件的元组执行投影运算，得到最终的结果元组

□ 可以简写为： $\pi_A \sigma_F(R)$

➤ 在没有括号的情况下，其运算顺序为：从右向左

综合举例：单个关系上的 元组选择 与 属性投影

□ 例2.11：查询选修了2号课程的学生学号。

$$\pi_{Sno}(\sigma_{Cno = '2'}(SC)) = \{201215121, 201215122\}$$

选修关系 SC

Sno	Cno	Grade
201215121	1	92
201215121	2	85
201215121	3	88
201215122	2	90
201215122	3	80

$\sigma_{Cno = '2'}$

选择运算执行结果

Sno	Cno	Grade
201215121	2	85
201215122	2	90

π_{Sno}

投影结果

Sno
201215121
201215122

关系代数的基本运算（总结）

运算	运算前提	结果关系模式	结果关系
$R \cup S$	R和S必须是同类关系	与关系R（或关系S）的模式相同	$\{t \mid t \in R \vee t \in S\}$
$R - S$			$\{t \mid t \in R \wedge t \notin S\}$
$R \times S$	任意两个关系	由关系R和关系S中的所有属性组成	$\{(t_r, t_s) \mid t_r \in R \wedge t_s \in S\}$
$\sigma_F(R)$	选择条件F是一个只能由关系R中的属性名、常量、比较运算符、逻辑运算符组成的逻辑表达式	与关系R的模式相同	$\{t \mid t \in R \wedge F(t) = true\}$
$\pi_A(R)$	属性组A只能是关系R的一个属性子集	由属性组A构成结果关系的模式	$\{t[A] \mid t \in R\}$

➤ 相容表/同类关系：两个关系的关系模式同时满足以下两个要求

- ①具有相同的目n（具有相同数目的属性） ②相应的属性来自于同一个域

关系代数基本运算符的优先级

	运算符		优先级
一元运算	$\sigma_F(R)$	$\pi_A(R)$	↓ 从高到低
笛卡尔积	$R \times S$		
集合运算	$R \cup S$	$R - S$	

2.4 关系代数（基本运算）

- 一. 导入：关系代数概述
- 二. 关系代数的基本运算
- 三. 关系代数基本运算的应用
- 四. 基本运算小结

三. 关系代数基本运算的应用

- ❑ MyCAP数据库
- ❑ 基本运算的查询语义
- ❑ 使用基本运算的查询表示

❑ 设有一个公司产品零售数据库，其关系模式如下(编号属性分别是各个关系的码)：

关系名	属性名	关系模式
顾客	顾客编号, 姓名, 居住城市, 折扣	Customers (cid, cname, city, discnt)
供应商	供应商编号, 名称, 所在城市, 佣金比例	Agents (aid, aname, city, percent)
商品	商品编号, 名称, 库存城市, 库存数量, 单价	Products (pid, pname, city, quantity, price)
订单	订单编号, 订购日期, 顾客编号, 供应商编号, 商品编号, 订购数量, 销售金额	Orders (ordno, month, cid, aid, pid, qty, dollars)

CUSTOMERS

<u>cid</u>	cname	city	discnt
c001	TipTop	Duluth	10.00
c002	Basics	Dallas	12.00
c003	Allied	Dallas	8.00
c004	ACME	Duluth	8.00
c006	ACME	Kyoto	0.00

AGENTS

<u>aid</u>	aname	city	percent
a01	Smith	New York	6
a02	Jones	Newark	6
a03	Brown	Tokyo	7
a04	Gray	New York	6
a05	Otasi	Duluth	5
a06	Smith	Dallas	5

PRODUCTS

<u>pid</u>	pname	city	quantity	price
P01	comb	Dallas	111400	0.50
p02	brush	Newark	203000	0.50
p03	razor	Duluth	150600	1.00
p04	pen	Duluth	125300	1.00
p05	pencil	Dallas	221400	1.00
p06	folder	Dallas	123100	2.00
p07	case	Newark	100500	1.00

ORDERS

<u>ordno</u>	month	cid	aid	pid	qty	dollars
1011	jan	c001	a01	p01	1000	450.00
1012	jan	c001	a01	p01	1000	450.00
1019	feb	c001	a02	p02	400	180.00
...	...	...	...	...	...	...

三. 关系代数基本运算的应用

❑ 关系代数基本运算的定义

- 每个运算的运算前提、运算结果（结果关系模式、结果元组的语义）

❑ 如何正确书写关系代数表达式

❑ 熟悉每一种运算符的使用场景和需要注意的要点

- 何时需要使用差运算？在使用差运算时需要注意什么？
- 单个关系上的数据查询：选择与投影运算的调用顺序
- 在选择运算中，何时 & 如何使用‘不等’比较
- 何时需要使用笛卡尔积？笛卡尔积的运算结果是什么？
- ‘赋值’运算的使用

基本运算的查询语义

【复习思考题20】公司产品零售数据库，其关系模式如下(带下划线的属性是码)：

customers (<u>cid</u> , cname, city, discent)	products (<u>pid</u> , pname, city, quantity, price)
agents (<u>aid</u> , aname, city, percent)	orders (<u>ordno</u> , month, cid, aid, pid, qty, dollars)

在下述六组关系代数查询表达式中，请分析他们各自查询的目标对象、查询语义和结果集的相互关系。

第1组	$Q_1: \sigma_{pid='p01'}(orders)$	$Q_2: \sigma_{pid \neq 'p01'}(orders)$
第2组	$Q_3: \pi_{cid}(\sigma_{pid='p01'}(orders))$	$Q_4: \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
第3组	$Q_5: \pi_{cid}(orders) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	$Q_6: \pi_{cid}(orders) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
第4组	$Q_7: \pi_{cid}(customers) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	$Q_8: \pi_{cid}(customers) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
第5组	$Q_9: orders - \sigma_{pid='p01'}(orders)$	$Q_{10}: orders - \sigma_{pid \neq 'p01'}(orders)$
第6组	$Q_{11}: \pi_{cid}(orders - \sigma_{pid='p01'}(orders))$	$Q_{12}: \pi_{cid}(orders - \sigma_{pid \neq 'p01'}(orders))$

思考题 (第1组)

第1组	$Q_1: \sigma_{pid = 'p01'}(orders)$	$Q_2: \sigma_{pid \neq 'p01'}(orders)$
-----	-------------------------------------	--

□ 查询结果集的语义是什么？

Q_1 : p01号商品的所有购买‘订单’

Q_2 : 除了p01号商品外，其他所有商品的购买‘订单’

□ 两个结果集的关系是什么？

$Q_1 \cap Q_2 = \text{空集}$

$Q_1 \cup Q_2 = \text{Orders}$

思考题 (第2组)

第2组	$Q_3: \pi_{cid}(\sigma_{pid = 'p01'}(orders))$	$Q_4: \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
-----	--	---

□ 查询结果集的语义是什么？

Q_3 : 购买过p01号商品的‘顾客(编号)’

Q_4 : 购买过其他商品(pid不等于p01)的‘顾客(编号)’

□ 两个结果集的关系是什么？

- 两个结果集之间的关系，取决于当前orders表中的订单情况
- 两个结果集之间可能有交集，也可能没有交集，甚至于一个结果集包含着另一个结果集，一切皆有可能！
- $Q_3 \cup Q_4 = \pi_{cid}(orders)$

思考题 (第3组)

第3组	Q ₅	$\pi_{cid}(\textit{orders}) - \pi_{cid}(\sigma_{pid = 'p01'}(\textit{orders}))$
	Q ₆	$\pi_{cid}(\textit{orders}) - \pi_{cid}(\sigma_{pid \neq 'p01'}(\textit{orders}))$

□ 查询结果集的语义是什么？

Q₅: 购买过商品但没有购买过p01号商品的‘顾客(编号)’

Q₆: 只购买过p01号商品的‘顾客(编号)’

□ 两个结果集的关系是什么？

$$Q_5 \cap Q_6 = \text{空集}$$

$$Q_5 \subseteq Q_4$$

$$Q_4: \pi_{cid}(\sigma_{pid \neq 'p01'}(\textit{orders}))$$

$$Q_5 \cup Q_6 \subseteq \pi_{cid}(\textit{orders})$$

$$Q_6 \subseteq Q_3$$

$$Q_3: \pi_{cid}(\sigma_{pid = 'p01'}(\textit{orders}))$$

思考题 (第4组)

第4组	Q_7	$\pi_{cid}(customers) - \pi_{cid}(\sigma_{pid = 'p01'}(orders))$
	Q_8	$\pi_{cid}(customers) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$

□ 查询结果集的语义是什么？

Q_7 : 没有购买过p01号商品的顾客 (包括“没有购买过任何商品”的顾客)

Q_8 : “只购买过p01号商品”的顾客 和 “没有购买过任何商品”的顾客

□ 两个结果集的关系是什么？

根据顾客的商品购买情况，将所有顾客划分为以下互不相交的四个子集：

- 没有购买过任何商品的顾客集合 S_1
- 只购买过p01这一种商品的顾客集合 S_2
- 没有购买过p01但购买过其他商品的顾客集合 S_3
- 既购买过p01又购买过其他商品的顾客集合 S_4

则: $Q_7 = S_1 \cup S_3$

$Q_7 \cap Q_8 = S_1$

$Q_8 = S_1 \cup S_2$

$Q_7 \cup Q_8 = S_1 \cup S_2 \cup S_3$

思考题 (第5组)

第5组	Q_9	$orders - \sigma_{pid = 'p01'}(orders)$
	Q_{10}	$orders - \sigma_{pid \neq 'p01'}(orders)$

□ 查询结果集的语义是什么？

Q_9 : 除了p01号商品之外，其他商品的所有销售订单

Q_{10} : p01号商品的所有销售订单

□ 两个结果集的关系是什么？

$Q_9 \cap Q_{10} = \text{空集}$

$Q_9 \cup Q_{10} = orders$

$Q_9 = Q_2$

$Q_{10} = Q_1$

$Q_1: \sigma_{pid = 'p01'}(orders)$
$Q_2: \sigma_{pid \neq 'p01'}(orders)$

(注: orders表中的pid不可能取空值)

思考题 (第6组)

第6组	Q_{11}	$\pi_{cid}(orders - \sigma_{pid = 'p01'}(orders))$
	Q_{12}	$\pi_{cid}(orders - \sigma_{pid \neq 'p01'}(orders))$

□ 查询结果集的语义是什么？

Q_{11} : 购买过其他商品 (pid不等于'p01') 的顾客编号的集合

Q_{12} : 购买过商品'p01'的顾客编号的集合

□ 两个结果集的关系是什么？

就如同前面的 Q_3 和 Q_4 一样, Q_{11} 和 Q_{12} 的结果集之间也没有确定的相互关系, 一切取决于当前orders表中的订单情况。

- Q_{11} 是“购买过其他商品”的顾客集合, 其中的顾客有的可能也购买过'p01', 也有的可能没有购买过'p01';
- Q_{12} 是“购买过商品p01”的顾客集合, 其中的顾客可能也购买过其他商品, 也可能没有购买过其他商品。

思考题 -- 总结 & 思考 (注: '其他商品' 是指商品编号pid不等于'p01'的商品)

	查询表达式	目标对象	目标对象需要满足的条件
Q ₁	$\sigma_{pid='p01'}(orders)$	订单	'p01'号商品的销售订单
Q ₂	$\sigma_{pid \neq 'p01'}(orders)$	订单	其他商品的销售订单
Q ₃	$\pi_{cid}(\sigma_{pid='p01'}(orders))$	顾客	该顾客购买过'p01'号商品
Q ₄	$\pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$	顾客	该顾客购买过其他商品
Q ₅	$\pi_{cid}(orders) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	顾客	该顾客没有购买过'p01', 但购买过其他商品
Q ₆	$\pi_{cid}(orders) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$	顾客	该顾客只购买过'p01', 但没有购买过其他商品
Q ₇	$\pi_{cid}(customers) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	顾客	该顾客没有购买过'p01'号商品 (请注意与Q ₅ 的区别)
Q ₈	$\pi_{cid}(customers) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$	顾客	该顾客没有购买过其他商品 (请注意与Q ₆ 的区别)
Q ₉	$orders - \sigma_{pid='p01'}(orders)$	订单	其他商品的销售订单
Q ₁₀	$orders - \sigma_{pid \neq 'p01'}(orders)$	订单	'p01'号商品的销售订单
Q ₁₁	$\pi_{cid}(orders - \sigma_{pid='p01'}(orders))$	顾客	购买过其他商品的顾客 (请注意与Q ₅ 的区别)
Q ₁₂	$\pi_{cid}(orders - \sigma_{pid \neq 'p01'}(orders))$	顾客	购买过'p01'号商品的顾客 (请注意与Q ₆ 的区别)

□ 求解过程

- 1) 确定查询目标（确定结果关系的关系模式）
- 2) 明确查询条件（确定结果元组的语义）
- 3) 选择从条件到目标的查找路径，并据此确定操作对象，即：
 - 在操作过程中需要使用到那些关系？
 - 这些关系又是如何联合起来实现查询的？
- 4) 根据以上分析构造关系代数表达式

使用基本运算的查询表示

1. 查询所有顾客所居住的城市名称；
2. 查询所有居住在京都的顾客；
3. 查询存放于达拉斯且单价超过\$0.50的商品；
4. 查询所有折扣低于10%的顾客所在城市以及佣金低于6%的代理商所在城市的城市名称；
5. 查询位于同一个城市的顾客和代理商，给出顾客的姓名、代理商的名称以及他们所在城市的名称；
6. 查询姓名为‘Allied’的顾客的订购信息，列出其每一条订单记录所购买商品的商品编号、订购日期以及订购数量；
7. 查询顾客、代理商以及所订购的商品都位于同一个城市的订单的编号；
8. 查询没有销售过‘p02’号商品的供应商的编号；
9. 查询只销售过‘p02’号商品的供应商的编号；
10. 在当前的所有顾客中，查询折扣最高的顾客的编号。

Example 1

1. 查询所有顾客所居住的城市名称

Customers

<u>cid</u>	cname	city	discent
c001	TipTop	Duluth	10.00
c002	Basics	Dallas	12.00
c003	Allied	Dallas	8.00
c004	ACME	Duluth	8.00
c006	ACME	Kyoto	0.00

$\pi_{city}(Customers)$

city
Duluth
Dallas
Kyoto

❑ 在结果关系中，要剔除重复的 'Duluth' 及 'Dallas' 元组，保证结果关系中元组的唯一性。

Example 2

2. 查询所有居住在京都 (Kyoto) 的顾客

Customers

<u>cid</u>	cname	city	discnt
c001	TipTop	Duluth	10.00
c002	Basics	Dallas	12.00
c003	Allied	Dallas	8.00
c004	ACME	Duluth	8.00
c006	ACME	Kyoto	0.00

$\sigma_{city = 'Kyoto'} (Customers)$

cid	cname	city	discnt
c006	ACME	Kyoto	0.00

Example 3

3. 查询存放于达拉斯(Dallas)且单价(price)超过\$0.50的商品

Products

<u>pid</u>	pname	city	quantity	price
P01	comb	Dallas	111400	0.50
p04	pen	Duluth	125300	1.00
p05	pencil	Dallas	221400	1.00
p06	folder	Dallas	123100	2.00
p07	case	Newark	100500	1.00

$\sigma_{city = 'Dallas' \wedge price > 0.50} (Products)$

<u>pid</u>	pname	city	quantity	price
p05	pencil	Dallas	221400	1.00
p06	folder	Dallas	123100	2.00

- 为了方便后续关系代数表达式的表示，将各个关系的关系名可简写成其首字母：

CUSTOMERS → **C**

AGENTS → **A**

PRODUCTS → **P**

ORDERS → **O**

Example 4

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

4. 查询所有折扣(discnt)低于10%的顾客所在城市以及佣金(percent)低于6%的代理商所在城市的城市名称

$$\pi_{city}(\sigma_{discnt < 10}(C)) \cup \pi_{city}(\sigma_{percent < 6}(A))$$

【思考】能否将该查询表示如下？

$$\pi_{city}((\sigma_{discnt < 10}(C)) \cup (\sigma_{percent < 6}(A)))$$

【答】不能。中间的并运算 $(\sigma_{discnt < 10}(C)) \cup (\sigma_{percent < 6}(A))$ 是不合法的。

Example 5 (1)

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

5. 查询位于同一个城市的顾客和代理商，给出顾客的姓名(cname)、代理商的名称(aname)以及他们所在城市的名称(city)

$$\pi_{C.cname, C.city, A.aname} (\sigma_{C.city=A.city} (C \times A))$$

□ 说明：

- ① 先分析用户查询要求，确定本次查询需要使用到的关系：**顾客和代理商**（本次查询不要求在目标顾客和代理商之间存在订单）
- ② 先用笛卡尔积实现顾客和代理商关系的合并，再用选择运算查出符合条件的笛卡尔积结果元组，最后用投影运算生成目标关系。

Example 5 (2)

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

5. 查询位于同一个城市的顾客和代理商，给出顾客的姓名(cname)、代理商的名称(aname)以及他们所在城市的名称(city)

$\pi_{C.cname, C.city, A.aname} (\sigma_{C.city=A.city} (C \times A))$

【思考】请用关系代数的基本运算来表示下述查询：

- ① 查询位于同一个城市且相互之间存在订单的顾客和代理商，给出顾客的姓名、代理商的名称、他们所在城市的名称；
- ② 查询位于同一个城市且相互之间不存在订单的顾客和代理商，给出顾客的姓名、代理商的名称、他们所在城市的名称。

Example 6 (1)

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

6. 查询姓名为‘Allied’的顾客的订购信息，列出其每一条订单记录所购买商品的商品编号(pid)、订购月份(month)以及订购数量(qty)

① 查询表达式 (一)

$$\pi_{O.pid, O.month, O.qty} (\sigma_{C.cid=O.cid \wedge C.cname='Allied'} (C \times O))$$

② 查询表达式 (二)

$$\pi_{O.pid, O.month, O.qty} (\sigma_{C.cid=O.cid} (\sigma_{C.cname='Allied'} (C) \times O))$$

Example 6 (2)

6. 查询姓名为‘Allied’的顾客的订购信息，列出其每一条订单记录所购买商品的商品编号(pid)、订购月份(month)以及订购数量(qty)

① 查询表达式 (一)

$$\pi_{O.pid, O.month, O.qty} (\sigma_{C.cid=O.cid \wedge C.cname='Allied'} (C \times O))$$

② 查询表达式 (二)

$$\pi_{O.pid, O.month, O.qty} (\sigma_{C.cid=O.cid} (\sigma_{C.cname='Allied'} (C) \times O))$$

③ 查询表达式 (三)

$$\pi_{O.pid, O.month, O.qty} (\sigma_{C.cid=O.cid} (\pi_{cid} (\sigma_{C.cname='Allied'} (C)) \times O))$$

□ 说明:

- 可提前执行单个关系上的元组选择查询，也可添加部分投影操作，以降低中间计算结果的数据量；
- 添加新的投影操作，不能影响后续运算的合法性。

Example 7

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

7. 查询顾客、代理商以及所订购的商品都位于同一个城市的订单编号

➤ 查询表达式 (一)

$$\pi_{ordno} (\sigma_{C.city=A.city \wedge P.city=A.city \wedge C.cid=O.cid \wedge A.aid=O.aid \wedge P.pid=O.pid} (C \times A \times P \times O))$$

➤ 查询表达式 (二)

$$\pi_{ordno} (\sigma_{C.city=A.city \wedge P.city=A.city \wedge C.cid=O.cid \wedge A.aid=O.aid \wedge P.pid=O.pid} (\pi_{cid, city} (C) \times \pi_{aid, city} (A) \times \pi_{pid, city} (P) \times O))$$

Example 8

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

8. 查询没有销售过 'p02' 号商品的供应商的编号 (aid) ;

❑ 可能出现的错误表示

➤ $\pi_{aid}(\sigma_{pid \neq 'p02'}(O))$

➤ $\pi_{aid}(\sigma_{pid = 'p02' \wedge qty = 0}(O))$

❑ 正确的查询表示如下

$$\pi_{aid}(A) - \pi_{aid}(\sigma_{pid = 'p02'}(O))$$

❑ 另一种错误表示: $\pi_{aid}(O) - \pi_{aid}(\sigma_{pid = 'p02'}(O))$

Example 9

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

9. 查询只销售过 'p02' 号商品的供应商的编号

❑ 可能出现的错误表示: $\pi_{aid}(\sigma_{pid='p02'}(O))$

❑ 正确的查询表示如下

$$\pi_{aid}(O) - \pi_{aid}(\sigma_{pid \neq 'p02'}(O))$$

❑ 本题不能表示为: $\pi_{aid}(A) - \pi_{aid}(\sigma_{pid \neq 'p02'}(O))$

Example 8 & 9

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

8. 查询没有销售过‘p02’号商品的供应商的编号(aid)

$$\pi_{aid}(A) - \pi_{aid}(\sigma_{pid = 'p02'}(O))$$

9. 查询只销售过‘p02’号商品的供应商的编号

$$\pi_{aid}(O) - \pi_{aid}(\sigma_{pid \neq 'p02'}(O))$$

【例】查询销售过‘p02’号商品的供应商的编号

$$\pi_{aid}(\sigma_{pid = 'p02'}(O))$$

□ 请仔细体会：何时需要使用‘差’运算？何时使用‘不等’比较？

Example 10 (1)

C (cno, cname, city, discnt)

P (pid, pname, city, quantity, price)

A (aid, aname, city, percent)

O (ordno, month, cid, aid, pid, qty, dollars)

10. 在当前的所有顾客中，查询折扣(discnt)最高的顾客的编号

<u>cid</u>	cname	city	discnt
<i>c001</i>	<i>TipTop</i>	<i>Duluth</i>	<i>10.00</i>
<i>c002</i>	<i>Basics</i>	<i>Dallas</i>	<i>12.00</i>
<i>c003</i>	<i>Allied</i>	<i>Dallas</i>	<i>8.00</i>
<i>c004</i>	<i>ACME</i>	<i>Duluth</i>	<i>8.00</i>
<i>c006</i>	<i>ACME</i>	<i>Kyoto</i>	<i>0.00</i>

Example 10 (2)

<u>cid</u>	cname	city	discnt
c001	TipTop	Duluth	10.00
c002	Basics	Dallas	12.00
c003	Allied	Dallas	8.00
c004	ACME	Duluth	8.00
c006	ACME	Kyoto	0.00

10. 在当前的所有顾客中，查询折扣(discnt)最高的顾客的编号(cid)

➤ 错误答案: $\pi_{C.cid}(\sigma_{discnt = MAX(discnt)}(C))$

□ 回顾: 选择运算 $\sigma_F(R)$

根据给定的条件表达式 F 来查询一个关系中有哪些元组满足查询要求，具体计算过程是：

- 针对关系 R 中的每一个元组 t ，将 t 中元组的属性值代入到条件表达式 F 中进行计算，如果结果为true，那么就说元组 t 能够使得条件 F 成立，则元组 t 满足查询要求并被加入到查询的结果元组集合中。
- 因此在选择条件表达式 F 中使用统计函数MAX是没有意义的（在单个元组上，对一个属性的取值进行最大值的统计计算是没有意义的）。
- 在部分参考资料中介绍了扩充关系代数操作‘聚集’，但也仅用于对查询结果集进行统计计算。

Example 10 (3)

<u>cid</u>	cname	city	discnt
<i>c001</i>	<i>TipTop</i>	<i>Duluth</i>	<i>10.00</i>
<i>c002</i>	<i>Basics</i>	<i>Dallas</i>	<i>12.00</i>
<i>c003</i>	<i>Allied</i>	<i>Dallas</i>	<i>8.00</i>
<i>c004</i>	<i>ACME</i>	<i>Duluth</i>	<i>8.00</i>
<i>c006</i>	<i>ACME</i>	<i>Kyoto</i>	<i>0.00</i>

□ 对于一个顾客c，如何判定c的折扣是“最高” or “不是最高”？

- a) 如果c的折扣不小于所有顾客的折扣，则c的折扣就是最高的；
- b) 如果能够找到一个顾客g，g的折扣比c的折扣高，那么c的折扣就不是最高的。

Example 10 (4)

<u>cid</u>	cname	city	discnt
<i>c001</i>	<i>TipTop</i>	<i>Duluth</i>	<i>10.00</i>
<i>c002</i>	<i>Basics</i>	<i>Dallas</i>	<i>12.00</i>
<i>c003</i>	<i>Allied</i>	<i>Dallas</i>	<i>8.00</i>
<i>c004</i>	<i>ACME</i>	<i>Duluth</i>	<i>8.00</i>
<i>c006</i>	<i>ACME</i>	<i>Kyoto</i>	<i>0.00</i>

□ 给定一个顾客c，如何判定c的折扣“不是最高”？

b) 如果能够找到一个或一些顾客g，g的折扣比c的折扣要高，那么c的折扣就不是最高的。

□ 换句话说，给定一个顾客c，用条件 **c.discnt < g.discnt** 在顾客关系上进行元组选择，查找满足上述要求的顾客g。如果结果集不为空，则c就是折扣不是最高的顾客。

Example 10 (5)

<u>cid</u>	cname	city	discnt
<i>c001</i>	<i>TipTop</i>	<i>Duluth</i>	<i>10.00</i>
<i>c002</i>	<i>Basics</i>	<i>Dallas</i>	<i>12.00</i>
<i>c003</i>	<i>Allied</i>	<i>Dallas</i>	<i>8.00</i>
<i>c004</i>	<i>ACME</i>	<i>Duluth</i>	<i>8.00</i>
<i>c006</i>	<i>ACME</i>	<i>Kyoto</i>	<i>0.00</i>

❑ 为了查询得到折扣不是最高的顾客，我们需要两个顾客集合（完全一样的关系*C*和关系*S*）

➤ 顾客关系*C*: 作为目标对象的候选集

➤ 顾客关系*S*: 作为条件对象的集合

❑ 通过这两个顾客关系，如何在*C*中找出折扣不是最高的顾客元组？

➤ 在关系*C*中查找折扣不是最高的顾客*c*，而关系*S*中顾客元组则是用来判断顾客*c*是不是‘折扣不是最高’。

➤ 也就是说，对于一个给定的顾客*c*，我们在关系*S*中查找折扣比*c*高的顾客元组*s*，如果能够找到这样的元组，那么顾客*c*就是满足条件的（折扣不是最高）。

Example 10 (6)

<u>cid</u>	cname	city	discnt
<i>c001</i>	<i>TipTop</i>	<i>Duluth</i>	<i>10.00</i>
<i>c002</i>	<i>Basics</i>	<i>Dallas</i>	<i>12.00</i>
<i>c003</i>	<i>Allied</i>	<i>Dallas</i>	<i>8.00</i>
<i>c004</i>	<i>ACME</i>	<i>Duluth</i>	<i>8.00</i>
<i>c006</i>	<i>ACME</i>	<i>Kyoto</i>	<i>0.00</i>

□ 令 $S := C$

$$R_2 := \pi_{C.cid} (\sigma_{C.discnt < S.discnt} (C \times S))$$

R_2

cid
<i>c001</i>
<i>c003</i>
<i>c004</i>
<i>c006</i>

- 计算结果如左表所示
- R_2 即为折扣不是最高的顾客编号的集合。

□ 思考：

- ① 如果顾客 *c006* 的折扣 *discnt* 不是 0 而是一个空值， R_2 的计算结果是什么？
- ② 如果顾客表中的所有顾客元组的 *discnt* 值都是空值， R_2 的计算结果是什么？

Example 10 (7)

- ❑ 为了完整地演示计算过程，我们仅保留顾客关系中的cid和discnt两个属性，以及c001,c002,c003三个元组。从而得到一个简化的顾客关系C和关系S，并演示如何去查询得到‘折扣最高’的顾客。

CUSTOMERS

<u>cid</u>	cname	city	discnt
<i>c001</i>	<i>TipTop</i>	<i>Duluth</i>	<i>10.00</i>
<i>c002</i>	<i>Basics</i>	<i>Dallas</i>	<i>12.00</i>
<i>c003</i>	<i>Allied</i>	<i>Dallas</i>	<i>8.00</i>
<i>c004</i>	<i>ACME</i>	<i>Duluth</i>	<i>8.00</i>
<i>c006</i>	<i>ACME</i>	<i>Kyoto</i>	<i>0.00</i>

精简后的关系 C

cid	discnt
<i>c001</i>	<i>10</i>
<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>

Example 10 (8)

C

cid	discnt
<i>c001</i>	<i>10</i>
<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>

S

cid	discnt
<i>c001</i>	<i>10</i>
<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>

$C \times S$

C.cid	C.discnt	S.cid	S.discnt
<i>c001</i>	<i>10</i>	<i>c001</i>	<i>10</i>
<i>c001</i>	<i>10</i>	<i>c002</i>	<i>12</i>
<i>c001</i>	<i>10</i>	<i>c003</i>	<i>8</i>
<i>c002</i>	<i>12</i>	<i>c001</i>	<i>10</i>
<i>c002</i>	<i>12</i>	<i>c002</i>	<i>12</i>
<i>c002</i>	<i>12</i>	<i>c003</i>	<i>8</i>
<i>c003</i>	<i>8</i>	<i>c001</i>	<i>10</i>
<i>c003</i>	<i>8</i>	<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>	<i>c003</i>	<i>8</i>

Example 10 (9)

$\sigma_{C.discnt < S.discnt} (C \times S)$

C.cid	C.discnt	S.cid	S.discnt
c001	10	c002	12
c003	8	c001	10
c003	8	c002	12

Example 10 (10)

$$\sigma_{C.discnt < S.discnt} (C \times S)$$

C.cid	C.discnt	S.cid	S.discnt
<i>c001</i>	<i>10</i>	<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>	<i>c001</i>	<i>10</i>
<i>c003</i>	<i>8</i>	<i>c002</i>	<i>12</i>

顾客关系 C

cid	discnt
<i>c001</i>	<i>10</i>
<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>

□ 在C.cid列上留下了“折扣不是最高”的顾客的cid

Example 10 (11)

$\sigma_{C.discnt < S.discnt} (C \times S)$

C.cid	C.discnt	S.cid	S.discnt
c001	10	c002	12
c003	8	c001	10
c003	8	c002	12

顾客关系 C

cid	discnt
c001	10
c002	12
c003	8

折扣不是最高的顾客cid

C.cid
c001
c003

□ 投影得到 “折扣不是最高” 的顾客的cid

$\pi_{C.cid} (\sigma_{C.discnt < S.discnt} (C \times S))$

Example 10 (12)

$\sigma_{C.discount < S.discount} (C \times S)$

C.cid	C.discount	S.cid	S.discount
c001	10	c002	12
c003	8	c001	10
c003	8	c002	12

顾客关系 C

cid	discount
c001	10
c002	12
c003	8

折扣不是最低的顾客cid

S.cid
c002
c001

□ 换个方式，可投影得到“折扣不是最低”的顾客的cid

$\pi_{S.cid} (\sigma_{C.discount < S.discount} (C \times S))$

Example 10 (13)

$$\sigma_{C.discnt < S.discnt} (C \times S)$$

C.cid	C.discnt	S.cid	S.discnt
<i>c001</i>	<i>10</i>	<i>c001</i>	<i>10</i>
<i>c001</i>	<i>10</i>	<i>c002</i>	<i>12</i>
<i>c001</i>	<i>10</i>	<i>c003</i>	<i>8</i>
<i>c002</i>	<i>12</i>	<i>c001</i>	<i>10</i>
<i>c002</i>	<i>12</i>	<i>c002</i>	<i>12</i>
<i>c002</i>	<i>12</i>	<i>c003</i>	<i>8</i>
<i>c003</i>	<i>8</i>	<i>c001</i>	<i>10</i>
<i>c003</i>	<i>8</i>	<i>c002</i>	<i>12</i>
<i>c003</i>	<i>8</i>	<i>c003</i>	<i>8</i>

□ **注意：** 在这一步的选择运算中，切记不能使用‘小于等于’比较！

Example 10 (14)

❑ 使用了‘小于等于’比较后，选择运算的执行结果，实现不了挑选‘折扣不是最高’或‘折扣不是最低’顾客的目标！
(在C.cid和S.cid列上，都含有所有顾客的cid)

$\sigma_{C.discount \leq S.discount} (C \times S)$

C.cid	C.discount	S.cid	S.discount
c001	10	c001	10
c001	10	c002	12
c002	12	c002	12
c003	8	c001	10
c003	8	c002	12
c003	8	c003	8

Example 10 (总结)

10. 在当前的所有顾客中，查询折扣(discont)最高的顾客的编号(cid)

① 查询所有顾客的编号，其结果构成关系 R_1

$$R_1 := \pi_{cid}(C)$$

② 查询折扣并非最高的顾客（其折扣低于其他某个顾客的折扣，或至少存在一个顾客X，而X的折扣高于当前顾客的折扣）的编号，查询结果构成关系 R_2

$$\text{令 } S := C, \text{ 则: } R_2 := \pi_{C.cid}(\sigma_{C.discount < S.discount}(C \times S))$$

③ 利用减法(difference)运算获得折扣最高顾客的编号: $T := R_1 - R_2$

□ 合并上述的三个公式，得到该查询的关系代数表达式如下：

$$\pi_{cid}(C) - \pi_{C.cid}(\sigma_{C.discount < S.discount}(C \times S))$$

[Example 10] $\pi_{cid}(C) - \pi_{C.cid}(\sigma_{C.discnt < S.discnt}(C \times S))$

【思考】 下述查询是否可以用关系代数来表示？

如果可以，请写出对应的关系代数表达式。

- ① 查询具有最小折扣的顾客的编号
- ② 查询折扣并非最大的顾客的编号
- ③ 查询其折扣为第二大的顾客的编号
- ④ 查询折扣等于所有顾客折扣的平均值的顾客的编号
- ⑤ 查询具有最大折扣的顾客的姓名(cname)

10. 具有最大折扣的顾客的编号: $S := C$

$$\pi_{cid}(C) - \pi_{C.cid}(\sigma_{C.discnt < S.discnt}(C \times S))$$

① 查询具有最小折扣的顾客的编号

② 查询折扣并非最大的顾客的编号

③ 查询其折扣为第二大的顾客的编号

➤ ①②③都可以用关系代数来表示 (课后思考)

④ 查询折扣等于所有顾客折扣的平均值的顾客的编号

➤ 在关系代数中没有引入统计功能, 第④题无法表示。

⑤ 查询具有最大折扣的顾客的姓名(cname)

➤ 第⑤题是否可以表示如下?

$$\pi_{cname}(C) - \pi_{C.cname}(\sigma_{C.discnt < S.discnt}(C \times S))$$

10. 具有最大折扣的顾客的编号: $S := C$

$$\pi_{cid}(C) - \pi_{C.cid}(\sigma_{C.discnt < S.discnt}(C \times S))$$

⑤ 查询具有最大折扣的顾客的姓名(cname)

➤ 第⑤题是否可以表示如下?

$$\pi_{cname}(C) - \pi_{C.cname}(\sigma_{C.discnt < S.discnt}(C \times S))$$

❑ 上述查询表示是错误的 (结果语义不符), 正确的查询表示如下

$$\pi_{cname}(\pi_{cid, cname}(C) - \pi_{C.cid, C.cname}(\sigma_{C.discnt < S.discnt}(C \times S)))$$

❑ 或者表示如下

➤ 令 $T := \pi_{cid}(C) - \pi_{C.cid}(\sigma_{C.discnt < S.discnt}(C \times S))$

➤ 本查询可表示为: $\pi_{C.cname}(\sigma_{C.cid = T.cid}(C \times T))$

❑ 要点: 在减数和被减数关系中, 必须含目标对象的码!

2.4 关系代数（基本运算）

- 一. 导入：关系代数概述
- 二. 关系代数的基本运算
- 三. 关系代数基本运算的应用
- 四. 基本运算小结

四. 关系代数基本运算小结 1

□ 关系代数基本运算的定义

- 并, 差, 笛卡尔积, 选择, 投影
- 每个运算的运算前提、运算结果 (结果关系模式、结果元组语义)

□ 关系代数基本运算的应用

- 如何正确书写关系代数表达式
- 熟悉每一种运算符的使用场景和需要注意的重点
 - 何时需要使用差运算? 在使用差运算时需要注意什么?
 - 单个关系上的数据查询: 选择与投影运算的调用顺序
 - 在选择运算中, 何时 & 如何使用 '不等' 比较
 - 何时需要使用笛卡尔积? 笛卡尔积的运算结果是什么?
 - '赋值' 运算的使用

四. 关系代数基本运算小结 2

运算	运算前提	结果关系模式	结果关系
$R \cup S$	R和S必须是同类关系	与关系R（或关系S）的模式相同	$\{ t \mid t \in R \vee t \in S \}$
$R - S$			$\{ t \mid t \in R \wedge t \notin S \}$
$R \times S$	任意两个关系	由关系R和关系S中的所有属性组成	$\{ (t_r, t_s) \mid t_r \in R \wedge t_s \in S \}$
$\sigma_F(R)$	选择条件F是一个只能由关系R中的属性名、常量、比较运算符、逻辑运算符组成的逻辑表达式	与关系R的模式相同	$\{ t \mid t \in R \wedge F(t) = true \}$
$\pi_A(R)$	属性组A只能是关系R的一个属性子集	由属性组A构成结果关系的关系模式	$\{ t[A] \mid t \in R \}$

➤ **同类关系**：两个关系的关系模式同时满足以下两个要求

- ①具有相同的目n（具有相同数目的属性） ②相应的属性来自于同一个域